

Lab 04 — Commented answers

Each answer follows the same pattern: the answer itself, the mechanism behind it, the nuance or pitfall, and an example you can verify at the terminal.

Question 1 — `COPY ../common/...`

Answer. The build can only see the **context** — the directory passed as argument (`.`, here `~/projects/api/`). `../common` lies outside it. Buildah clamps the path back inside the context (possible escaping context directory), finds nothing there, and fails. `-f` only selects the Dockerfile, not the boundary; an absolute path gets clamped into the context just the same; and `sudo` cannot help because this is a boundary problem, not a permissions problem. The fix: build from the parent directory (`podman build -f api/Dockerfile -t api:1.0 ~/projects`) with `COPY api/... common/...` paths, or copy `config.yml` into the project before building — or better still, keep it out of the image entirely and inject it at run time (lab 08).

Why. The context is a security and reproducibility boundary: a Dockerfile may only depend on what you hand it explicitly. Docker enforces this physically (the context is archived and sent to the daemon); Podman enforces it as a rule in Buildah — the result is the same.

Nuance. Podman supports multiple named contexts: `podman build --build-context common=../common .` followed by `COPY --from=common config.yml /app/`. That is the clean solution when a shared file genuinely belongs in several images.

Example.

```
podman build -f Dockerfile.out-of-context -t attempt .
# Error: building at STEP "COPY ../common/secret.txt /": ... possible escaping context directory error
```

Question 2 — `transferring context`, then nothing

Answer. Docker's client packs up the **entire** directory (1.1 GB) and sends it to the daemon before the first instruction runs: that is the `transferring context` line. Podman's Buildah reads the directory in place — no archive, no transfer — so the slowness vanishes. The second risk, however, is untouched: a `COPY . .` still bakes `node_modules` and `.git` **into the image** — 1.1 GB of dead weight, plus the full Git history (and any secrets buried in it) handed to anyone who pulls the image. `.dockerignore` therefore stays mandatory; it no longer buys speed, it protects the image's content.

Why. The context plays two roles: it is what gets *sent* (Docker only) and what is *copyable*. Podman eliminates the first cost, not the second.

Nuance. `.git` inside an image is a common and serious leak: the history often contains credentials that were removed "later". And without `.dockerignore`, a modified file inside `node_modules` also invalidates the `COPY` cache.

Example.

```
podman build -q -f Dockerfile.all -t t . # 218 MB image with node_modules
printf 'node_modules\n.git\n' > .dockerignore
podman build -q -f Dockerfile.all -t t2 . # 8.7 MB
```

Question 3 — EXPOSE and the port that does not answer

Answer. No, the image is fine. `EXPOSE` is a **declaration**: it documents that the application listens on 8080, and it feeds `podman ps` and `-P`. It creates no forwarding from the host. Without `-p 8080:8080`, the port is only reachable from the container network.

Why. Publishing a port is a deployment decision (which host port, which interface), not a property of the image. The image says "I listen on 8080"; the operator decides "I expose it on 18080".

Nuance. `-P` (capital) automatically publishes every `EXPOSE`d port on a random host port — that is where the declaration pays off. And in rootless mode, `-p 80:8080` fails (privileged port); pick a port ≥ 1024 or tune `net.ipv4.ip_unprivileged_port_start`.

Example.

```
podman run -d --name a my-api:1.0 && curl -m 2 localhost:8080      # failure
podman run -d --name b -p 8080:8080 my-api:1.0 && curl localhost:8080/actuator/health #
{"status":"UP"}
podman port b                                                    # 8080/tcp -> 0.0.0.0:8080
```

Question 4 — RUN java versus CMD java

Answer. A starts the API **during the build**: `RUN` executes its command at build time, so the API starts, never exits... and the build hangs (or, if the API does exit, the image gains nothing but a useless layer). B is correct: `CMD` records the command that `podman run` will launch.

Why. `RUN` prepares the file system (install, compile, copy); `CMD / ENTRYPOINT` describe the main process of the future container. Mixing them up means mixing up build time and run time.

Nuance. B would be better still with `ENTRYPOINT` + `CMD` (question 5) and a `USER`. And `RUN java -jar` does have a legitimate use: running a **finite task** at build time, such as `java -Djarmode=tools -jar app.jar extract` (lab 05).

Example.

```
podman build -f A -t a .      # STEP 3/3: RUN java -jar ... - never completes
podman build -f B -t b . && podman run -d -p 18080:8080 b
```

Question 5 — ENTRYPOINT + CMD versus CMD alone

Answer. A: `podman run img → java -jar /app/api.jar --spring.profiles.active=prod`; `podman run img --debug → java -jar /app/api.jar --debug` (the `CMD` is replaced, the `ENTRYPOINT` stays). B: `podman run img → the same full command`; `podman run img --debug → tries to run --debug by itself`, without `java`, and fails with `executable file not found`. Only B still allows `podman run img sh` — the whole `CMD` gets replaced by `sh`. With A, `podman run img sh` runs `java -jar api.jar sh`; you need `podman run --entrypoint sh img` instead.

Why. Arguments passed to `run` replace the `CMD` and are appended to the `ENTRYPOINT`. A suits an "application" image, B suits a "tool" image.

Nuance. In `exec` form, both A and B make `java` PID 1. A common variant at work: `ENTRYPOINT ["java", "-jar", "app.jar"]` with no `CMD`, and all configuration through environment variables — arguments are kept for debugging only.

Example.

```
timeout 5 podman run --rm api-lab:1.0 --debug | head -1      # Arguments recus : --debug
podman run --rm --entrypoint sh api-lab:1.0 -c 'echo ok'     # ok
```

Question 6 — Ten seconds, resorting to SIGKILL, no hooks

Answer. `CMD java -jar /app/api.jar` is a **shell** form: the engine runs `/bin/sh -c "java -jar /app/api.jar"`. On a Debian/Ubuntu base, `/bin/sh` is `dash`, which forks Java as a child and stays PID 1 itself. `podman stop` sends `SIGTERM` to PID 1 — the shell — which does not pass it on. Java never receives the signal and its *shutdown hooks* never run; after 10 seconds, Podman announces `resorting to SIGKILL` and kills everything (137). The fix: `CMD ["java", "-jar", "/app/api.jar"]`. On Alpine, `/bin/sh` is busybox's `ash`, which **replaces itself** with a simple command: Java becomes PID 1, receives `SIGTERM`, and the bug stays invisible in testing.

Why. A POSIX shell is under no obligation to forward signals to its children, and `dash` does not. The *exec* form removes the shell, and the problem with it.

Nuance. Even Alpine cannot rescue a shell `CMD` that contains `&&`, `|` or a variable: the shell then has to stick around. The rule "always use *exec* form" spares you from memorizing each shell's behavior.

Example.

```
podman exec s-deb ps -o pid,args | head -3      # 1 /bin/sh -c java ... 2 java ...
time podman stop s-deb                         # resorting to SIGKILL, 10 s, code 137
```

Question 7 — \$JAVA_OPTS not interpreted

Answer. In *exec* form there is **no shell**, so `$JAVA_OPTS` reaches Java untouched, as a literal six-character string. Two fixes. (1) Explicit shell form with `exec -- ENTRYPOINT ["sh", "-c", "exec java $JAVA_OPTS -jar /app/api.jar"]`; the cost is a dependency on `sh` and a less readable line. (2) Drop the variable — Java reads `JAVA_TOOL_OPTIONS` from the environment on its own, so use `ENV JAVA_TOOL_OPTIONS="-Xmx512m"` with `ENTRYPOINT ["java", "-jar", "/app/api.jar"]`; the cost is a `Picked up JAVA_TOOL_OPTIONS` message on `stderr` at start-up, and a variable that affects every Java process in the container.

Why. Expanding variables is a service the shell provides. The *exec* form is an argument array handed straight to the `execve` system call.

Nuance. Fix (1) without `exec` would bring back the problem from question 6. And for memory specifically, `-XX:MaxRAMPercentage=75` beats a fixed `-Xmx`: the JVM adapts to the cgroup (lab 10).

Example.

```
podman run --rm -e JAVA_TOOL_OPTIONS="-Xmx256m" api-lab:1.0 --debug 2>&1 | head -1
# Picked up JAVA_TOOL_OPTIONS: -Xmx256m
```

Question 8 — --build-arg and the secret

Answer. No. The `ARG` value does not appear in `Config.Env`, but it is recorded in the **history** of every instruction that uses it, and in the build cache. `podman history --no-trunc api:1.0` shows it

in plain text (`|1 DB_PASSWORD=Secr3t! /bin/sh -c ...`). Anyone who has the image has the password.

Why. The history records how each layer was produced, arguments included — that record is what makes caching possible. An `ARG` is an input to the build, therefore to the cache, therefore to the history.

Nuance. The sound practice: secrets have no business at *build* time. When one is unavoidable (a private Maven repository, say), `RUN --mount=type=secret,id=settings ...` exposes it during a single instruction without ever writing it to a layer (lab 08). A multi-stage build only protects you if the secret is used exclusively in a stage that gets thrown away (lab 05).

Example.

```
podman history --no-trunc api-lab:arg --format '{{.CreatedBy}}' | grep DB_PASSWORD
# |1 DB_PASSWORD=Secr3t! /bin/sh -c echo "build with $DB_PASSWORD" > /trace.txt
```

Question 9 — Six minutes per Maven build

Answer.

```
WORKDIR /app
COPY pom.xml .
RUN mvn -q dependency:go-offline      # dependency download, cached
COPY src ./src
RUN mvn -q package -DskipTests      # compilation only
```

The cache reuses an instruction when its text **and** its inputs are unchanged. `pom.xml` rarely changes, so the `dependency:go-offline` layer — the five minutes of downloading — stays on `Using cache`. When a `.java` file changes, only the compilation is replayed. The build stays slow whenever `pom.xml` changes — a dependency added or bumped — because that invalidates the dependencies layer.

Why. `COPY . /app` puts *all* the code ahead of Maven; every commit invalidates the copy, and with it everything downstream.

Nuance. `dependency:go-offline` is imperfect (some plugins still download during `package`). A `RUN --mount=type=cache,target=/root/.m2` (lab 05) keeps the local repository across builds, even when `pom.xml` changes. And with Podman, none of these gains needs a daemon: the cache lives in your user storage.

Example.

```
time podman build -t api-lab:2.1 .      # RUN ... sleep 5: --> Using cache; 0.7 s
```

Question 10 — Three apt `RUN`s

Answer. (1) **Three layers instead of one:** the apt lists (`/var/lib/apt/lists` , ~40 MB) land in layer 2; layer 3 merely hides them, so the image keeps carrying the 40 MB. (2) **An isolated `apt-get update`** gets cached: weeks later, a change to the `install` line will reuse stale indexes (packages not found, outdated versions). (3) **No `--no-install-recommends`**, and `vim` in a production image: tens of MB of unneeded packages, and that much more attack surface. The correct version:

```
RUN apt-get update \
&& apt-get install -y --no-install-recommends curl \
&& rm -rf /var/lib/apt/lists/*
```

Why. A layer is immutable; clean-up only takes effect in the layer that created the files. And the cache works instruction by instruction: `update` and `install` must travel together.

Nuance. On Alpine, `apk add --no-cache curl` does all of this in one line. And `vim` in an image is never justified: use `podman exec` with a temporary editor, or ship no editor at all (distroless image, lab 05).

Example.

```
podman history img --format 'table {{.Size}}\t{{.CreatedBy}}' # the "rm -rf" layer is 0B, the
one above 40 MB
```

Question 11 — `Using cache` then everything rebuilt

Answer. The rule: **an invalidated instruction invalidates every instruction after it**, and the cache is evaluated top to bottom. Day 1: only the last `COPY` changed and everything above it is identical → eight `Using cache` lines, then the last two steps rebuild. Day 2: inserting an instruction in third position changes the Dockerfile from line 3 onward → steps 3 through 10 are new to the cache and rebuild — even though their content did not change.

Why. A step's cache key is (parent layer, instruction, inputs). Change the parent layer and you change the key of everything below it.

Nuance. This is why volatile `ENV`, `ARG` and `LABEL` values (build number, date) belong **at the end** of a Dockerfile, while stable metadata goes early. An `ARG BUILD_DATE` on line 2 invalidates everything, on every single build.

Example.

```
podman build -t api-lab:3.1 . # STEP 3/5: COPY ... (replayed) then STEP 4/5: RUN ... sleep 5
(replayed too)
```

Question 12 — `ADD` versus `COPY`

Answer. Two behaviors specific to `ADD`: it automatically **unpacks** a local archive (`.tar`, `.tar.gz`, `.tar.xz`) into the destination, and it **downloads** URLs. The official recommendation is `COPY` because both behaviors are implicit: an `ADD file.tar.gz /app/` unpacks the archive when you meant to copy it, and a downloaded URL comes with no verification, no caching, and no way to `rm` it in the same layer. The one justified case: extracting a **local** archive in a single instruction (`ADD rootfs.tar.gz /`).

Why. A Dockerfile should read without surprises, and `COPY` does exactly one thing. For a URL, `RUN curl ... && tar ... && rm ...` in a single `RUN` is explicit and can clean up after itself.

Nuance. Both instructions accept `--chown` (and `--chmod`), which is handy before a `USER`. And `COPY --from=` (lab 05) has no `ADD` equivalent.

Example.

```
ADD app.tar.gz /opt/          # /opt/app/... unpacked
COPY app.tar.gz /opt/         # /opt/app.tar.gz as is
```

Question 13 — USER too early, and HUSER 100999

Answer. `USER` applies to every instruction that follows it, `RUN` included. Placed right after `FROM`, it makes `apt-get install` and `mkdir` run as UID 1000, which is not allowed to write to `/usr`, `/var` or `/`: hence `Permission denied`. In a well-written Dockerfile, `USER` goes **right before** `ENTRYPOINT` / `CMD`, after everything is installed, directories are created, and ownership is set (`chown`, `COPY --chown`). In rootless mode, the container's UID 1000 is mapped onto the host through `/etc/subuid`: the first "extra" UID (1) maps to 100000, so 1000 → 100999. `USER` still earns its place: (a) it takes root privileges away from the application *inside* the container (modifying image files, listening on port 80, installing packages); (b) the same image will run under Docker or Kubernetes, where root really is root; (c) security scanners and admission policies reject images that have no `USER`.

Why. The `user` namespace protects the *host*; `USER` protects the *container* and its contents. The two layers complement each other.

Nuance. `USER 1000:1000` works without creating the user (Podman even adds an `/etc/passwd` entry on the fly), but some programs expect a `HOME` or a name: `RUN adduser -D -u 1000 app` followed by `USER app` is more robust.

Example.

```
podman top u user,huser          # 1000 100999
podman run --rm --entrypoint sh api-lab:user-ok -c 'touch /app/x' # Permission denied: good.
```

Question 14 — "A single RUN"

Answer. The colleague is right whenever files created by one instruction are deleted by another (`install` + `clean-up`, `unpack` + `delete the archive`): split across layers, the files stay in the image. The colleague is wrong whenever the grouping mixes stable with volatile: a single `RUN` that downloads dependencies **and** compiles the code gets invalidated on every commit, so it downloads everything again; and a single 300 MB layer is re-uploaded in full on every `push`, whereas five layers — four of them stable — only transfer the delta.

Why. The layer count itself costs almost nothing. What matters is **which files live in which layer** (size) and **how often each layer changes** (cache and transfer).

Nuance. A practical rule: one `RUN` per "unit of change" — system packages (stable), dependencies (semi-stable), code (volatile). Multi-stage builds (lab 05) and Spring Boot's JAR layering apply exactly this logic.

Example.

```
podman history my-api:1.0 --format 'table {{.Size}}\t{{.CreatedBy}}' # one layer per unit of change
```